If you want to combine two or more hashes, the merge method is likely what you want. merge takes one or more hashes as arguments and an optional block. It returns a new hash, starting with the original receiving hash, and adds each key value pair from the arguments one by one. If it encounters a key that already exists in the hash and there is no block argument, the new value overwrites the old. If there is a block argument, then the block takes three arguments—the key, the existing value, and the new value—and the result of the block is added to the hash.

The merge! (also aliased as update) behaves the same except it changes the receiving hash in place, and also returns that hash:

```
hash_1 = {a: 1, b: 2, c: 3}
hash_2 = {c: 5, d: 4, e: 6}
hash_1.merge(hash_2)                                  # => {:a=>1, :b=>2, :c=>5,
                                                      # .. :d=>4, :e=>6}
hash_1.merge(hash_2) { |key, old, new| old + new } # => {:a=>1, :b=>2, :c=>8,
                                                      # .. :d=>4, :e=>6}
```

## Querying a Hash

There are several ways to query a hash for information. The length method, aliased as size, returns the number of key/value pairs in the hash. The empty? method reruns true if there are no key/value pairs in the hash.

If you want to find out if a particular key is in the hash, you'd use key?, which is aliased as has_key?, include?, or member?, but key? is the most common. It takes one argument and returns true if that argument is in the list of keys.

Going the other way, if you want to know whether a hash has a particular value no matter what the key, you'd use the value? method, aliased as has_value?.

Hashes also have a separate version of any?. Rather than taking a block, it takes a two-element array (that is, a key and a value) and returns true if that key value pair is in the hash.

## Iterating Hashes

When iterating over hashes in Ruby, it is important to remember that the order of the hash is consistent and based on the order in which the key/value pairs were added to the hash. So, the first element to be added will be first any time you iterate over the hash using any of the hash iteration methods, the second element added will be second, and so on.

Hashes implement each slightly differently than arrays in that the block argument to each takes two arguments: the key and the value. Then the block is called on each key/value pair in turn. This method is also aliased as each_pair.

You can integrate with one-argument blocks using each_key, which iterates over all the keys in order, or each_value, which iterates over all the values in order. Or you can just get an array of all the keys in order with keys or all the values in order with values.

## Comparing Hashes

Two hashes are equal (==) if they have the same keys and the value at each key is equal. The order of the keys does not matter; it only matters that the value matches.